Project #2

Mimicking of DB2.x-Firmware

Stein Lysberg

May 1, 2026

Contents

1 Introduction

This report is the second in a series of five, about the making of a plastic scanner. This report looks at the code written by the makers of the plastic scanner PCB (referred to as the Arduino code) [1], and maps out how its functionality can be implemented using a nRF52 DK running Zephyr Real Time Operating System (RTOS). Some technology and design choices made will also be addressed.

1.1 The Arduino code

Figure 1 shows a simple model of the flow of the code written for Arduino. After powering up, it gets ready to communicate to the connected computer via UART, as well as the on-board chips via I^{2}C. Following this, it performs a set-up routine for the LED-driver chip, and the ADC chip. It then calibrates the ADC, before it idles and awaits a command. It uses a Command-Line Interface (CLI) to control the plastic scanner from the computer terminal, which lets a user tell it what to do in real-time. The code written for this application will mimic the Arduino code, but differ in some ways, mainly in the exclusion of the CLI.

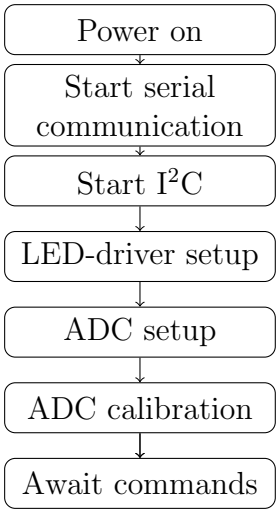

Figure 1: Arduino workflow chart

1.2 Technologies

This section looks at some of the technologies used in the plastic scanner, namely the Zephyr RTOS, I^{2}C, and the devicetree.

1.2.1 Zephyr RTOS

Zephyr RTOS is an open source real time operating system made for embedded devices, with emphasis on microcontrollers [2]. It uses the C language, and is Nordic Semiconductors recommended way of programming an nRF52 DK.

1.2.2 I^{2}C

I^{2}C (Inter-Integrated Circuit) is a standard for data transmission between devices, across two wires. One wire transmits data, and one has the clock. There can be multiple devices on the same I^{2}C-bus, and these are distiguished by having different addresses. This standard is commonly used for relatively cheap and small applications, and over small distances [3]. In this application it is used for communication between the microcontroller on the nRF52 DK, and the analog-to-digital converter (ADC) as well as the LED-driver.

1.2.3 Devicetree

The devicetree is a data structure to keep track of how hardware is connected, and how to communicate with it. Its purpose is to ease the job of making the same code usable in different environments [4]. For the microcontroller to communicate with the ADC and LED-driver via I^{2}C, they must be added to the device tree.

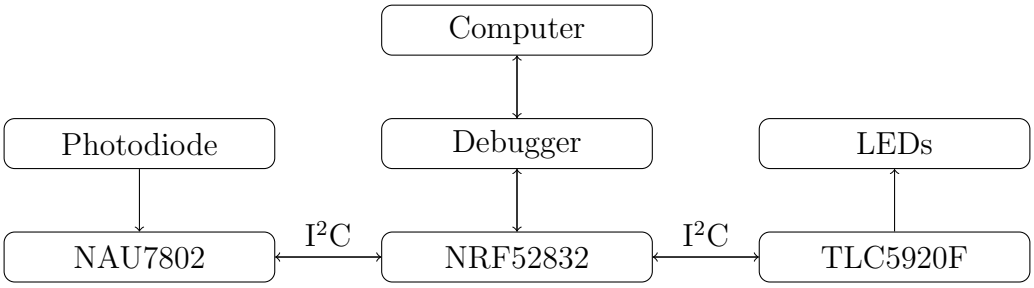

Figure 2: System architecture

1.3 Hardware and code

Figure 2 shows a simple schematic of how hardware is connected in this application. The microcontroller, a NRF52832, communicates to the computer via another NRF52832 called the *debugger* in the nRF52 DK. The microcontroller also communicates with the chips on the PCB, namely the NAU7802and the TLC5920F, using I^{C}C. Figure 3 shows the flow of the code written for the nRF52 DK. It differs from the Arduino code (fig. 1), mainly in that there is no use of a command line interface. Once it is powered up and has done its setup routine, it simply loops while printing the values from its scans.

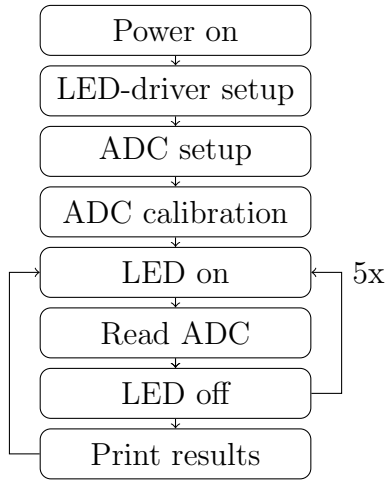

Figure 3: Workflow chart

1.3.1 LED-driver

This project uses a Texas Instruments TLC59208F as a LED-driver. In fig. 3, setup of the LED-driver means establishing communication using I²C, and then appropriately configuring the device for use in this application.

1.3.2 ADC

A Nuvoton NAU7802 is used as an ADC in this project. As with the LED-driver, setup means establishing communication, and configuration. In addition to this, the ADC should be calibrated using its own internal calibration routine, before it can be used. Otherwise, it might produce unreliable results [5].

1.3.3 The loop

Once the LED-driver and ADC are ready, the code enters a loop. First the first LED is turned on, then a reading is requested from the ADC. After making sure the reading is ready, it is retrieved and stored, and then the first LED is turned off again. This process is repeated an additional five times for LEDs 2-6, after which the results are printed to the console. The loop then starts over with the first LED.

2 Discussion

This section will discuss two of the choices which have been made for this project, first the choice between using Arduino and Zephyr RTOS, and secondly whether it is sufficient to use a simple loop rather than a command line interface.

2.1 Arduino vs. Zephyr RTOS

The Arduino programming language is based on C++, but writes like its own language. It is made to be easy to use, and there are abundant resources available for it online. Although there are Arduino products marketed towards proffesionals [6], it is largely considered a platform for education and hobby projects. Code for Zephyr RTOS is written in C. It is more used in industrial applications, and not made to be as user-friendly as Arduino. Zephyr RTOS is a more appropriate platform for this project than Arduino, because it is more geared towards proffesional and industrial application. It presents more of a challenge, and in return offers a higher level of control.

2.2 Loop vs. CLI

The Arduino code uses a command line interface to control the plastic scanner, while the code for this project simply loops, without the ability to receive input (as shown in figs. 1 and 3). Using a command line interface is the better solution, because it allows for more control while the device is in use, and allows for expansion of the functionality by adding more commands down the line. The code for this project was made to loop because it is easier to implement and sufficient for the purposes of this project. Other ways to control the plastic scanner can be made later, if needed, but a simple loop is sufficient for now.

3 Conclusion

This report has looked at the start of a process to mimic the plastic scanner code written for Arduino, and how it will be implemented using Zephyr RTOS on the nRF52 development board. Some of the technologies involved have been discussed, such as I^2C and the devicetree. The workflow of the Arduino code has also been investigated, and compared to the workflow which will be implemented in this project, with special attention to whether or not to include a command line interface. This brief report presents the foundation on which the rest of the project will be built.

References

[1] J. de Vos, "DB 2.3," 2023, last accessed 13 May 2025. [Online]. Available: https://docs.plasticscanner.com/boards/DB2.3

[2] Wikipedia contributors, "Zephyr (operating system) — Wikipedia, the free encyclopedia," 2025, last accessed 14 May 2025. [Online]. Available: https://en.wikipedia.org/wiki/Zephyr_(operating_system)

[3] ——, "I²c — Wikipedia, the free encyclopedia," 2025, last accessed 14 May 2025. [Online]. Available: https://no.wikipedia.org/wiki/I%C2%B2C

[4] ——, "Devicetree — Wikipedia, the free encyclopedia," 2025, last accessed 14 May 2025. [Online]. Available: https://en.wikipedia.org/wiki/Devicetree

[5] Nuvoton, "NAU7802," 2023, last accessed 13 May 2025. [Online]. Available: https://www.nuvoton.com/export/resource-files/en-us--DS_NAU7802_DataSheet_EN_Rev2.6.pdf

[6] Arduino, "Arduino PRO: Edge IoT technology," 2025, last accessed 15 May 2025. [Online]. Available: https://www.arduino.cc/pro/